Memory Efficient Doubly Linked List
–insertAfterElement and Time Complexity – Part 2

When next = NULL

–It occurs when Element is found and data is entered at end of the list, which have already elaborated.

When next ≠ NULL

–It occurs when Element is found but data is not enter at end of the list.

```
    while (curr != nullptr && curr->data != element)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
    }
```

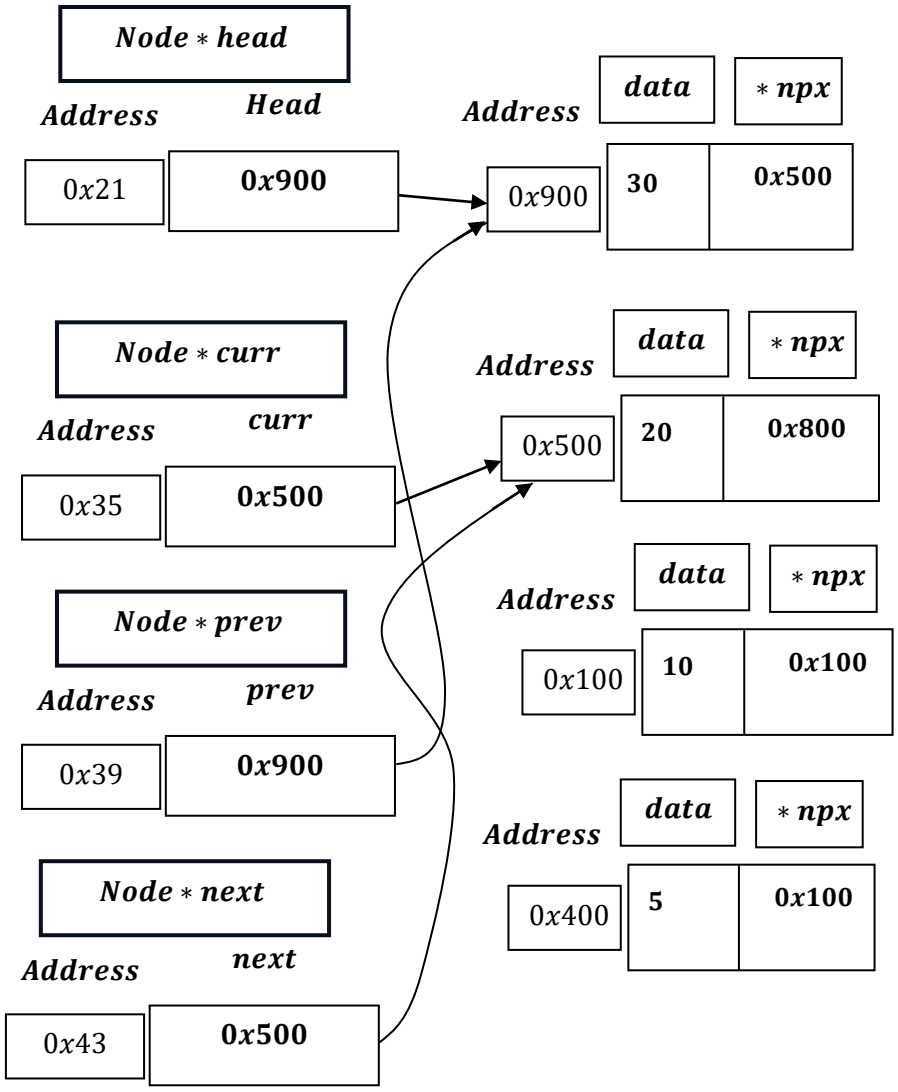
Lets say , Element = 10 and data = 15;

$\boldsymbol{curr: 0x900 \neq nullptr: 0x000\ AND\ curr \rightarrow data = 30\ \neq element = 10}$:

$\boldsymbol{next = XOR(prev: 0x000, curr \rightarrow npx: 0x500) = 0x000 \oplus 0x500 = 0x500.}$

$\boldsymbol{prev = curr = 0x900}$

$\boldsymbol{curr = next = 0x500}$

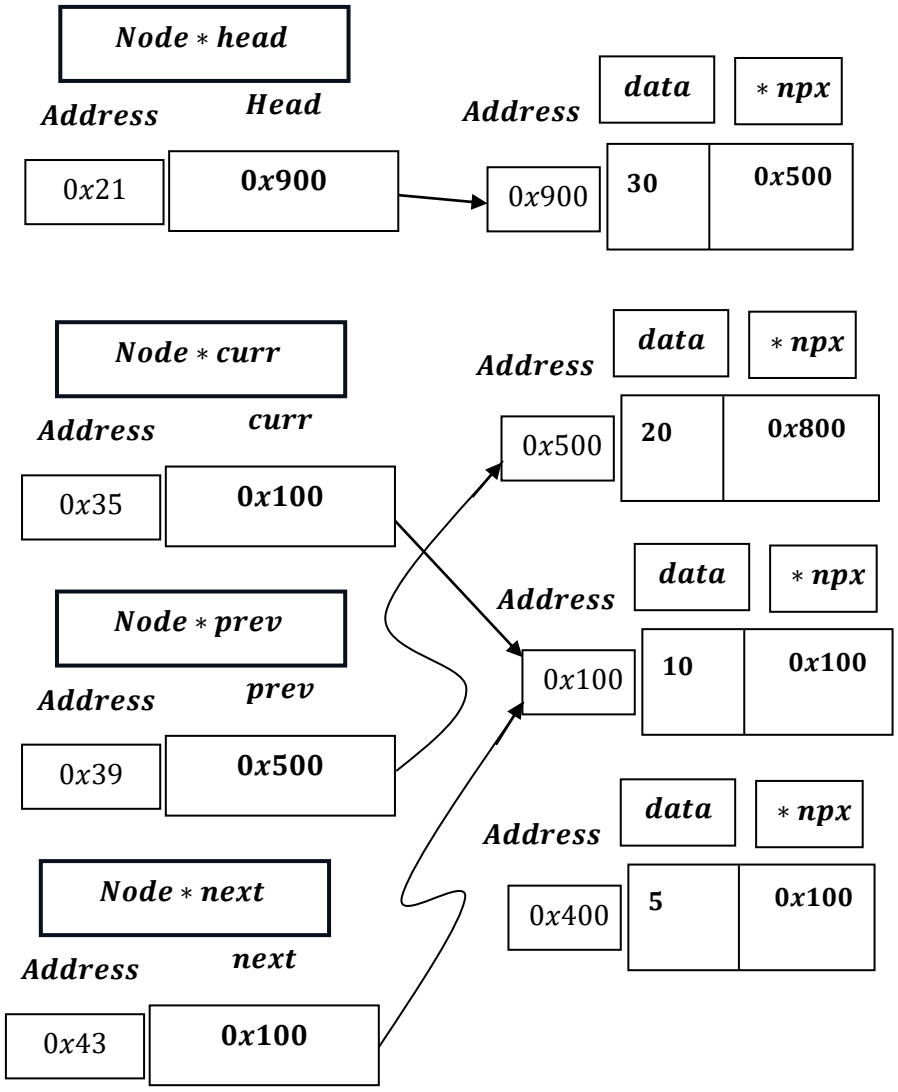


$curr: 0x500 \neq nullptr: 0x000\ AND\ curr \rightarrow data = 20\ \neq element = 10$:

$next = XOR(prev: 0x900, curr \rightarrow npx: 0x800) = 1001_2 \oplus 1000_2 = 0001_2 = 0x100$

$prev = curr = 0x500$

$curr = next = 0x100$

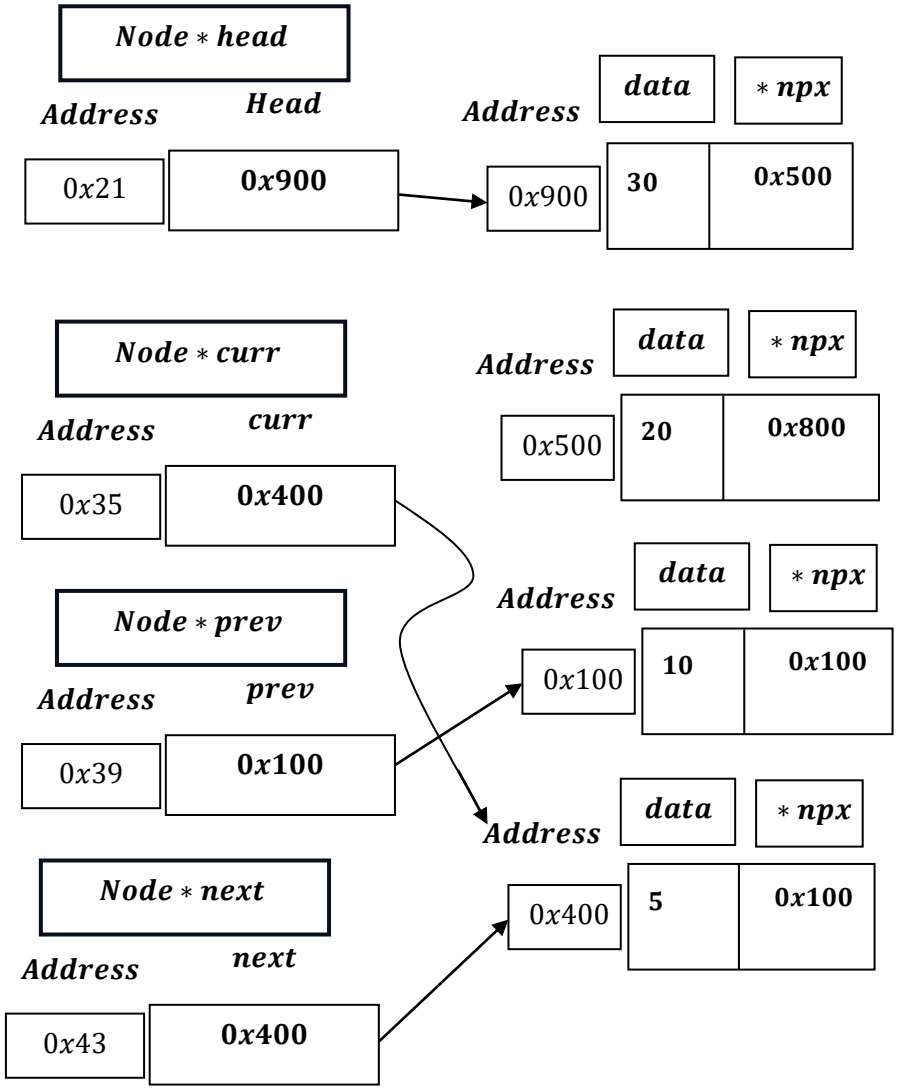


$curr: 0x100 \neq nullptr: 0x000[is\ TRUE]\ AND\ curr \rightarrow data = 10\ \neq element = 10[is\ False] \Longrightarrow Condition\ is\ False.$

```
    next = XOR(prev, curr->npx);
```

$next = XOR(prev: 0x500 \oplus curr \rightarrow npx: 0x100);$

$0101_2 \oplus 0001_2 = 0100_2 = 4_{10} = 0x400.$




```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members*:

→ *data(an integer) is typically 4 bytes.*

→ *npx(a pointer to another Node) is typically 8 bytes.*

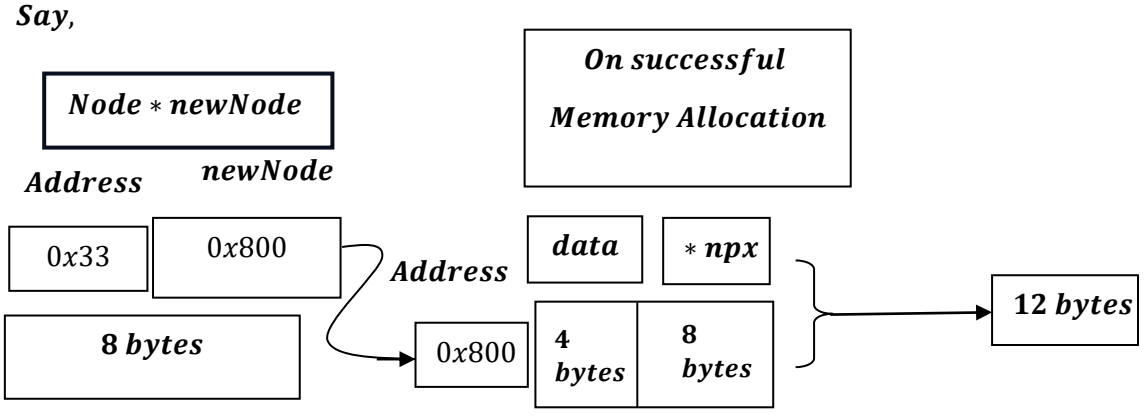
→ *Therefore , the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for npx) .*

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

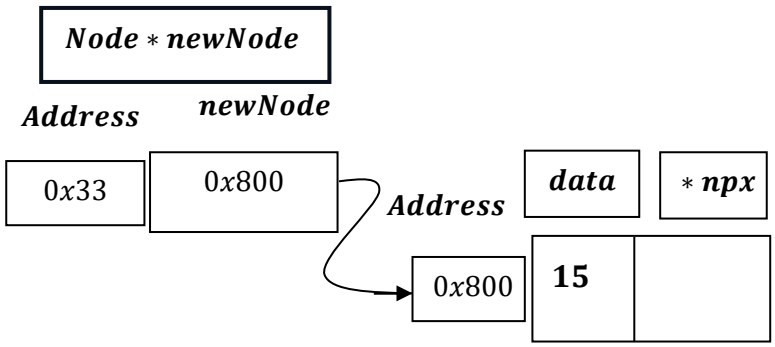
→ *newNode will try to allocate memory for data i.e. 4 bytes , npx pointer variable i.e. typically 8 bytes, i.e. total 12 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.*

Say,



```
newNode->data = data;
```

$\boldsymbol{Say, data = 15. Then, newNode \rightarrow data = data = 15.}$



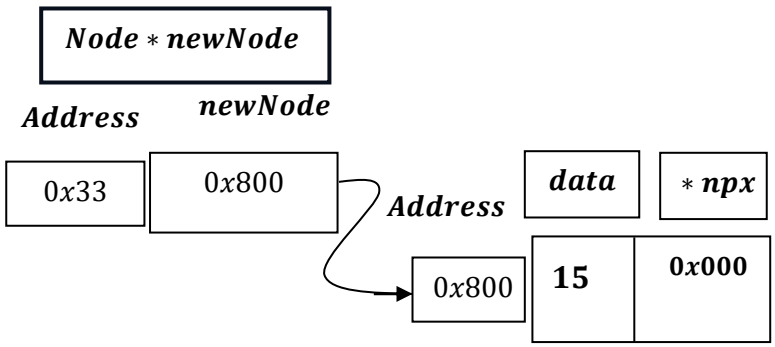

```
    newNode->npx = XOR(curr, next);
    curr->npx = XOR(prev, newNode);
```

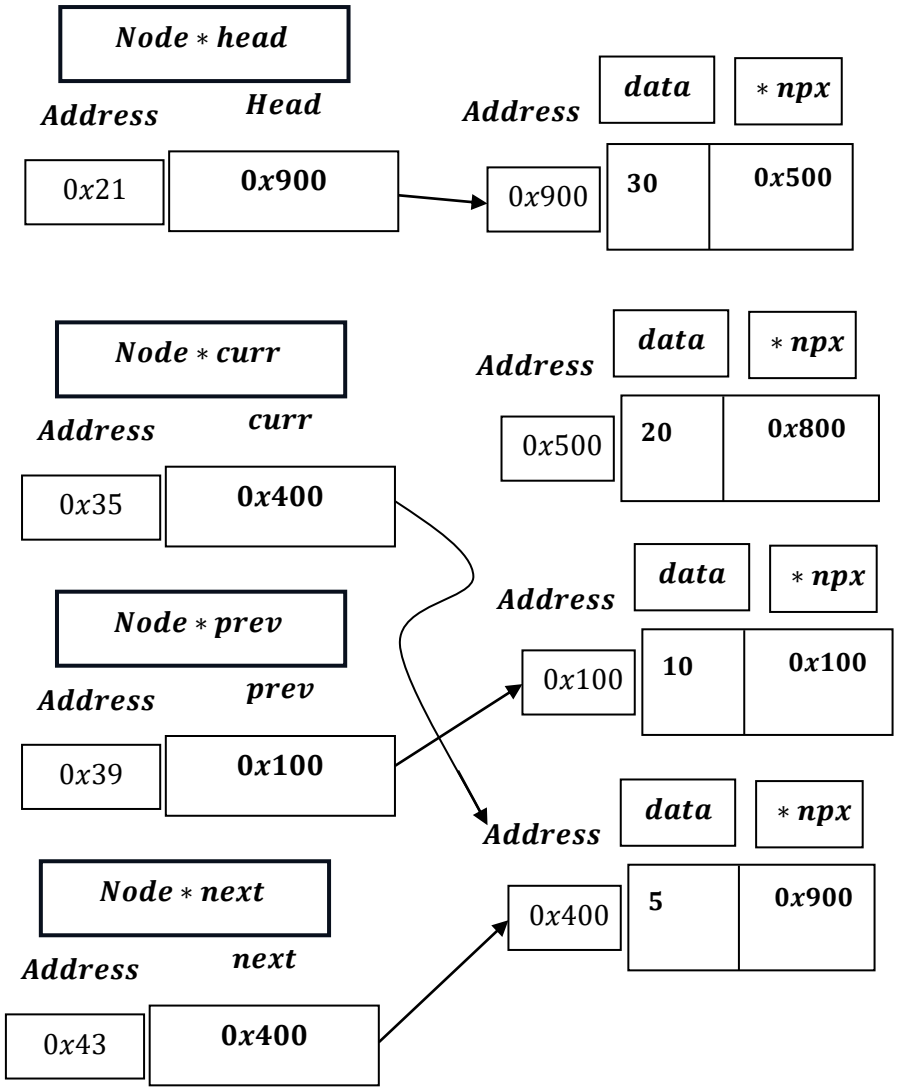
$\boldsymbol{newNode \rightarrow npx = XOR(curr: 0x400, next: 0x400)}$

$\boldsymbol{= 0100_2 \oplus 0100_2 = 000_2 = 0_{10} = 0x000}$.

$\boldsymbol{curr \rightarrow npx = XOR(prev, newNode) = (prev: 0x100, newNode: 0x800) =}$

$\boldsymbol{0001_2 \oplus 1000_2 = 1001_2 = 0x900.}$






```
if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(newNode, nextNext);
}
```

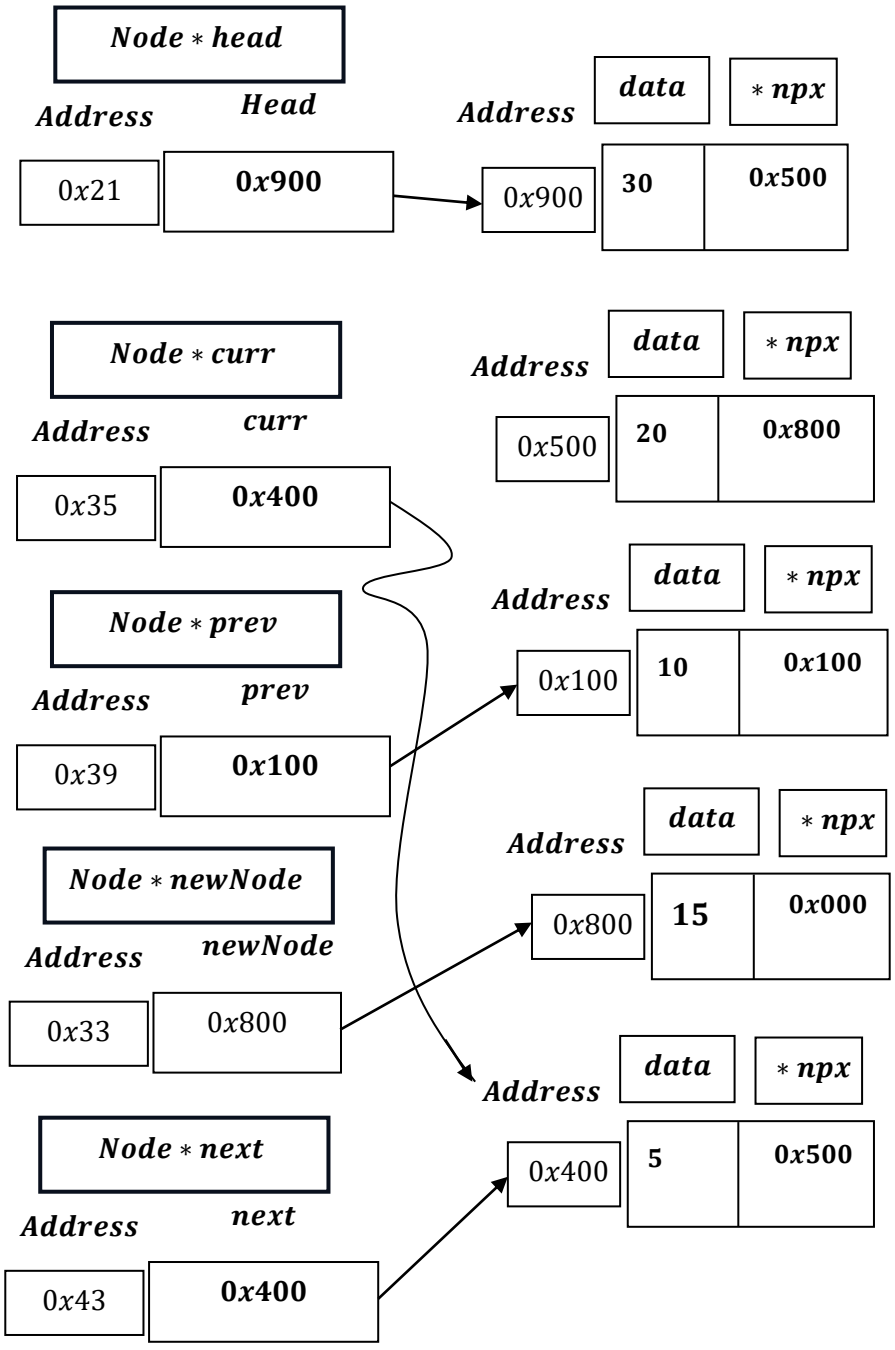
$$next \neq nullptr\ is\ TRUE:$$

$$Node * nextNext = XOR(curr: 0x400, next \rightarrow npx: 0x900)$$

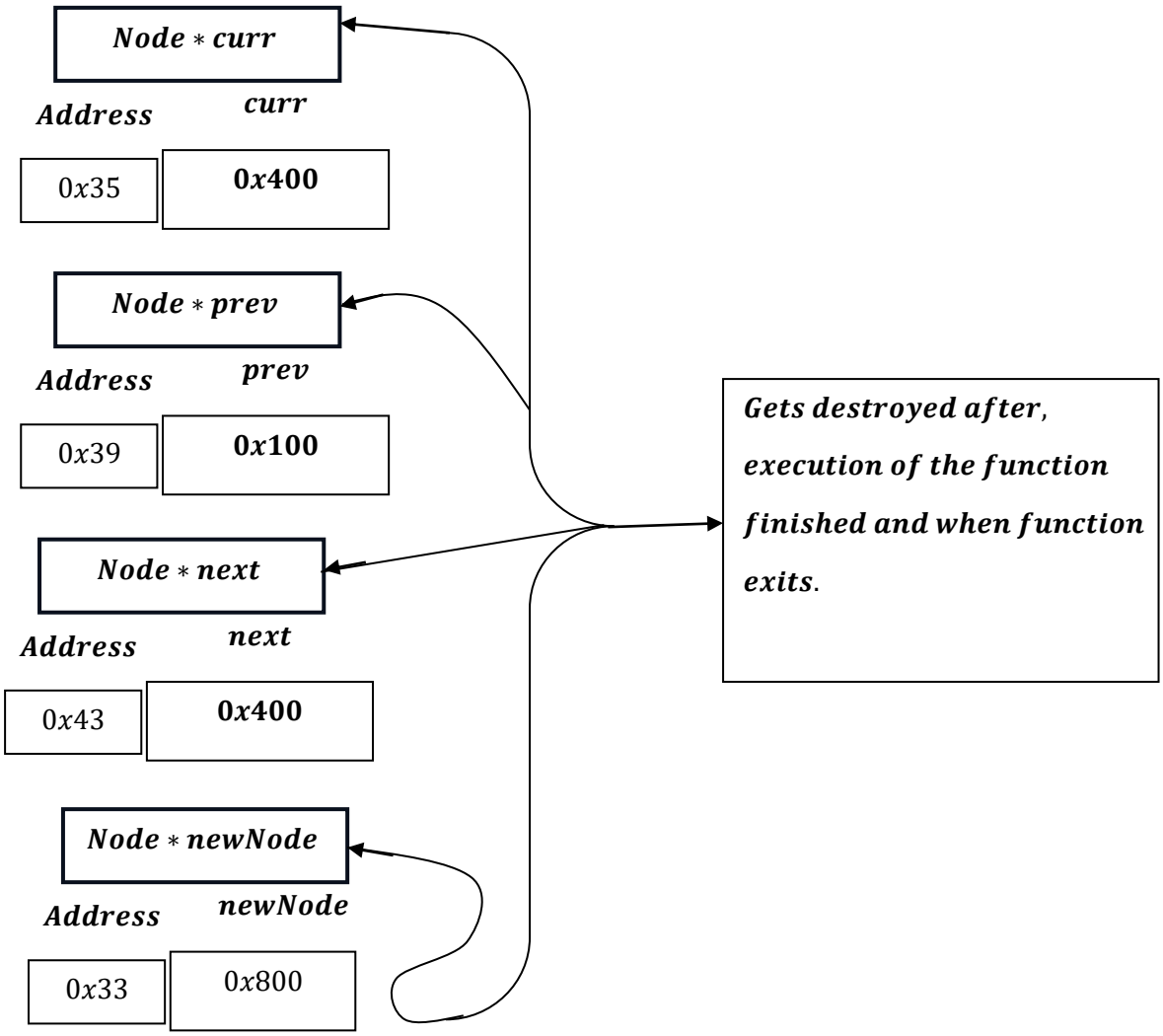
$$= 0100_2 \oplus 1001_2 = 1101_2 = 13_{10} = 0xD00$$

$$next \rightarrow npx = XOR(newNode: 0x800, nextNext: 0xD00)$$

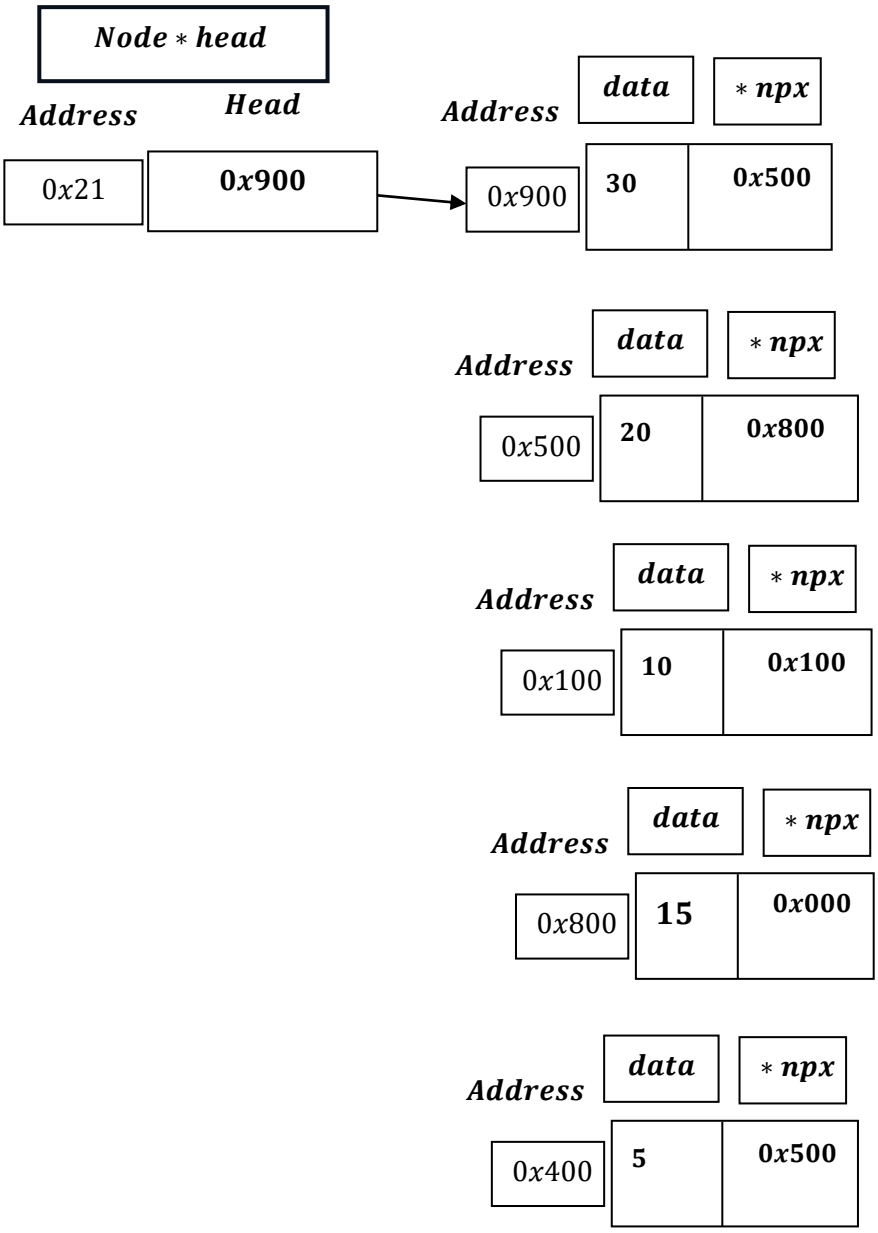
$$= 1000_2 \oplus 1101_2 = 0101_2 = 5_{10} = 0x500$$



As , functions ends:



And we are left with:


Node * head
Address
Head
0x21
0x900
Address
data
* npx
0x900
30
0x500
Address
data
* npx
0x500
20
0x800
Address
data
* npx
0x100
10
0x100
Address
data
* npx
0x800
15
0x000
Address
data
* npx
0x400
5
0x500